

파이썬 기본 문법 핵심 요약 정리

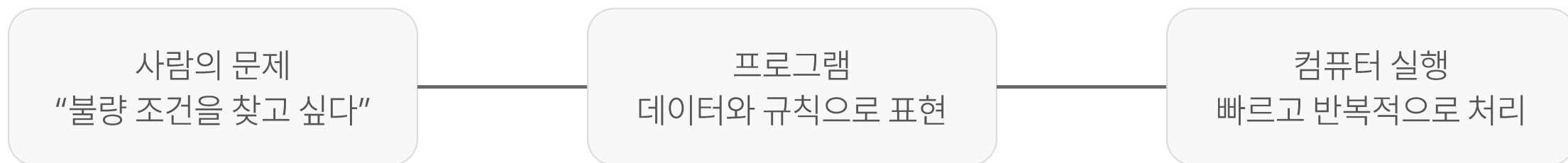
개념 먼저 · 간단 예제 중심 · 실전 친화적 설명

왜 프로그래밍을 배우는가?

프로그래밍은 컴퓨터에게 일을 시키는 방법

컴퓨터는 스스로 문제를 이해하지 않습니다.

사람이 문제를 정의하고, 컴퓨터가 실행할 수 있는 절차로 바꾸어 주어야 합니다.



문법 학습의 목표는 "코드 암기"가 아니라
문제를 컴퓨터가 처리할 수 있는 형태로 바꾸는 사고방식입니다.

프로그램은 데이터 처리 절차다

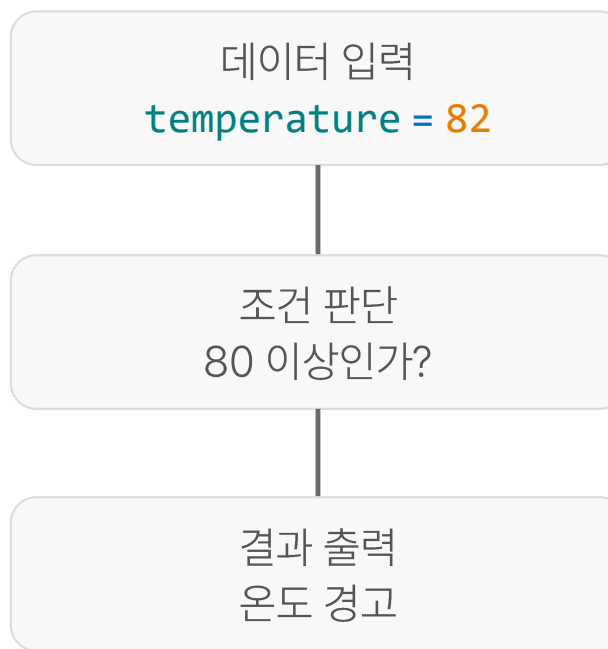
대부분의 코드는 입력 → 처리 → 출력의 흐름

예시: 생산 온도 판정

```
temperature = 82

if temperature >= 80:
    print("온도 경고")
else:
    print("정상")
```

이 코드가 하는 일



조건문, 반복문, 함수는 모두 이 절차를 더 정확하고 재사용 가능하게 표현하기 위한 도구입니다.

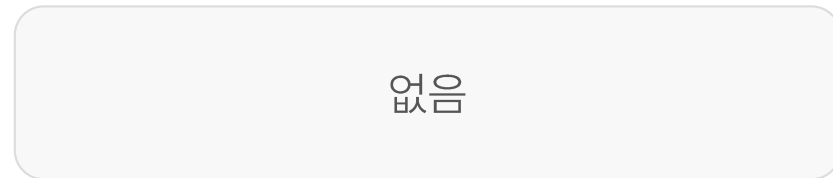
전통적 프로그래밍과 AI 프로그래밍의 차이

Software 1.0

전통적 프로그래밍



AI를 이용한 프로그래밍



전통적 프로그래밍과 AI 프로그래밍의 차이

Software 2.0: 전통적 방식은 사람이 규칙을 만들고, AI 방식은 데이터에서 규칙을 추정합니다.

전통적 프로그래밍



AI를 이용한 프로그래밍



데이터와 정답이 코드 그 자체라면?

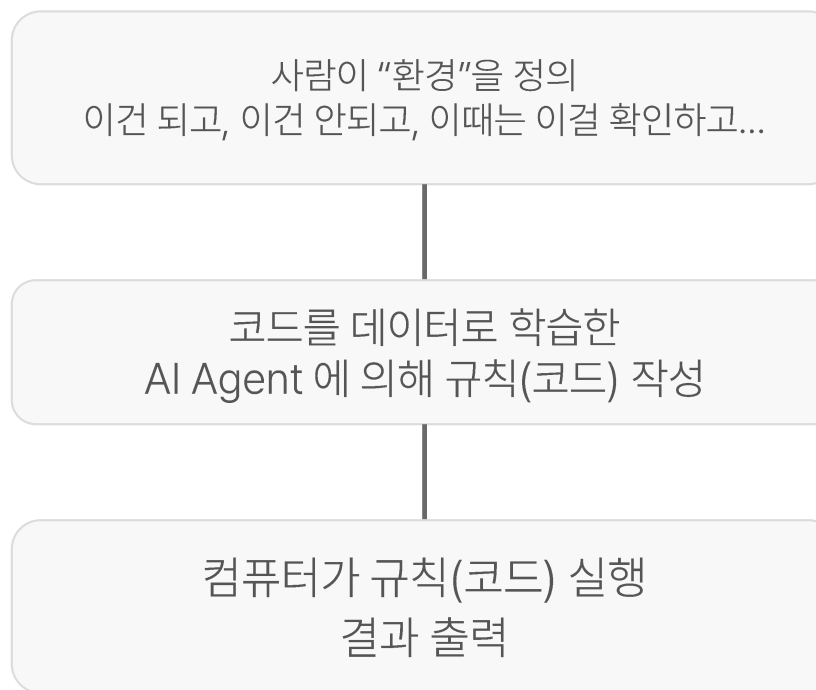
전통적 프로그래밍과 AI 프로그래밍의 차이

Software 3.0: AI 방식은 "해줘!"

전통적 프로그래밍



AI를 이용한 프로그래밍



AI 시대에도 파이썬 문법이 필요한 이유

AI가 코드를 만들어도, 사람이 코드를 읽고 판단할 수 있어야 함

AI가 잘 도와주는 일

코드 초안 작성
기존 코드 분석
오류 수정 제안
반복 작업 자동화
데이터 분석 코드 생성

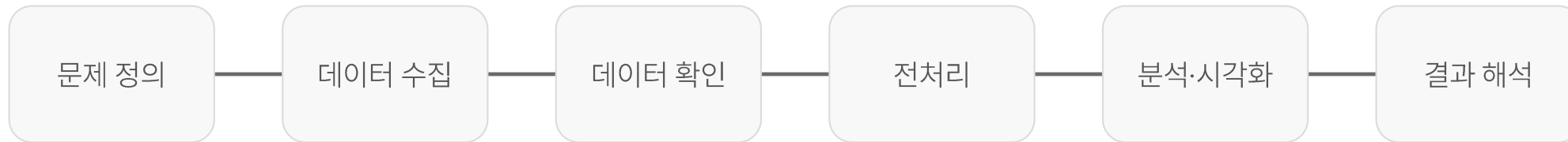
사람이 반드시 해야 하는 일

문제 정의
데이터 의미 이해
결과 검증
업무 맥락 반영

변수 · 조건문 · 반복문 · 함수의 의미를 모르면
AI가 만든 코드가 맞는지 판단하기 어렵습니다.

파이썬을 이용한 데이터 분석 프로세스

데이터 분석은 정해진 흐름을 따라 진행



예시 불량이 자주 발생하는 조건은 무엇인가?
온도, 압력, 속도, 시간 데이터를 확인하고 공통 패턴을 찾습니다.
필요하면 규칙 기반 판단 또는 머신러닝 모델로 이상 가능성을 예측합니다.

오늘 배우는 파이썬 문법은 이 전체 과정에서 데이터를 다루기 위한 기초 체력입니다.

프로그램 언어의 개념 총정리

프로그래밍 언어는 사람의 생각을 컴퓨터가 실행할 수 있는 명령으로 바꾸는 도구입니다.

값

숫자 · 문자 · 참/거짓

변수

값을 저장하는 이름

연산자

값을 계산하거나 비교하는 기호

조건문

상황에 따라 다른 명령 실행

반복문

같은 작업을 여러 번 실행

함수

자주 쓰는 코드를 묶어 재사용

라이브러리

미리 만들어진 기능 모음

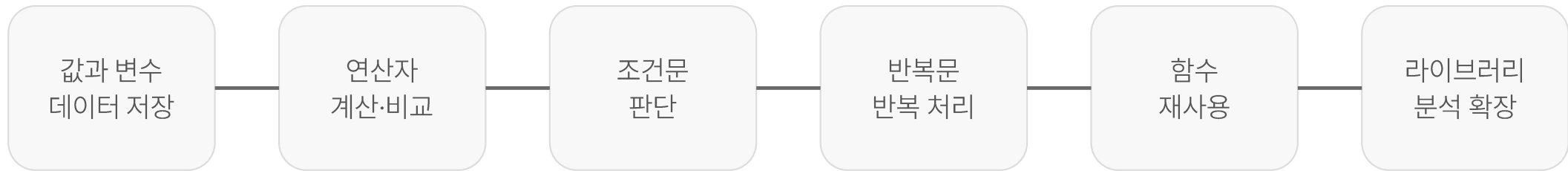
데이터 분석

문법을 실제 문제 해결에 적용

파이썬 학습 흐름: 값과 변수 → 연산자 → 조건문 → 반복문 → 함수 → 라이브러리 → 데이터 분석

이제 문법을 이렇게 연결해서 배웁니다

각 문법은 따로 떨어진 지식이 아니라 데이터 처리 절차의 한 부분입니다.



수업 중 예제는 먼저 일반적인 문법 예제로 익히고,
이후 간단한 생산·품질 데이터 처리 과제로 연결합니다.

핵심은 "코드를 실행한다"가 아니라 "문제를 처리 가능한 절차로 바꾼다"입니다.

Miniconda

- Miniconda is a free minimal installer for conda.
- <https://www.anaconda.com/download/success?reg=skipped>

Choose Your Download

Windows

Mac

Linux

Anaconda Distribution

Package and environment management with conda, Anaconda Navigator desktop app, and 600+ packages. Everything you need for data science.

GRAPHICAL INSTALLER

[↓ Windows 64-Bit Graphical Installer](#)

[📘 Graphical install instructions](#)

COMMAND LINE

[📄 CLI install instructions](#)

Miniconda

Lightweight installer that only includes conda, Python, and their dependencies. Install only what you need.

GRAPHICAL INSTALLER

[↓ Windows 64-Bit Graphical Installer](#)

[📘 Graphical install instructions](#)

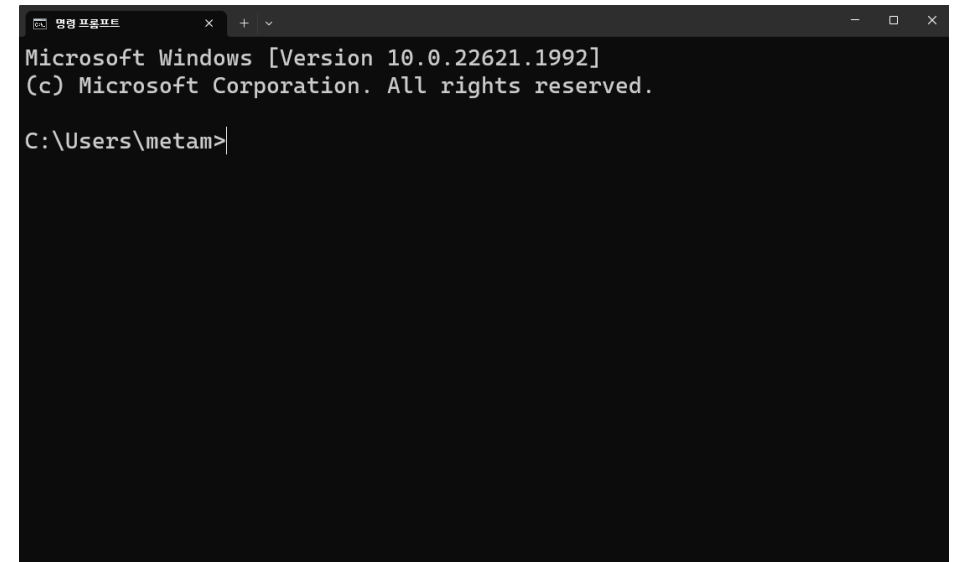
COMMAND LINE

[📄 CLI install instructions](#)

Need help choosing an installer? [Compare Anaconda Distribution vs. Miniconda](#) →

CMD 명령 프롬프트

- 파이썬과 파이썬 소스 코드를 실행하는 명령 입력창
- CMD 명령어
 - `dir` 또는 `ls`: 현재 디렉토리에 파일 목록을 출력
 - `cd` 디렉토리 이름: 현재 위치를 해당 디렉토리(폴더)로 바꿈
 - `mkdir` 디렉토리 이름: 현재 디렉토리에 해당 디렉토리를 생성
 - `rmdir` 디렉토리 이름 : 해당 디렉토리 삭제
- 특수 디렉토리 이름
 - `.` : 현재 디렉토리
 - `..` : 한 단계 상위 디렉토리

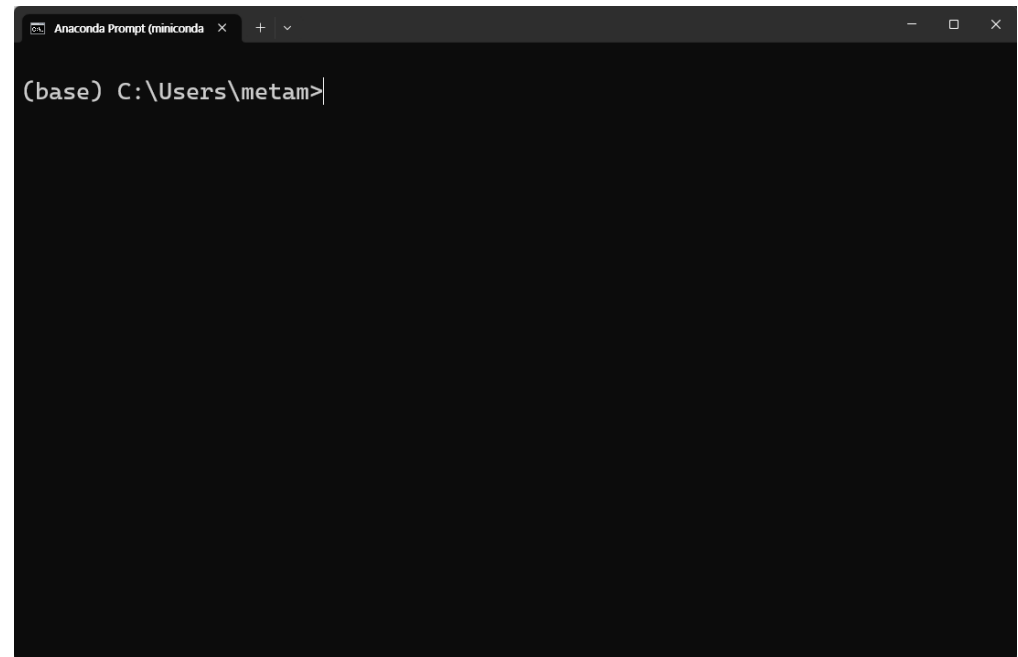


```
Microsoft Windows [Version 10.0.22621.1992]
(c) Microsoft Corporation. All rights reserved.

C:\Users\metam>
```

Anaconda 프롬프트

- 아나콘다 전용 CMD
 - 가상 환경이 기본적으로 활성화
 - conda 명령어 사용 가능



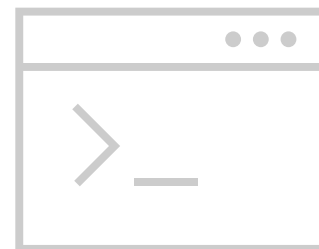
AI 에이전트 시대와 TUI

- 핵심 메시지

- TUI는 복고가 아니라, AI 에이전트와 함께 일하기 좋은 단순하고 일관된 인터페이스

- 배경

- GUI는 화려하지만 앱마다 조작 방식이 달라 사용자는 "어디를 눌러야 하지?"를 계속 생각
- TUI는 텍스트·키보드 중심 — 명령 입력 → 결과 확인 → 수정의 흐름이 빠르고 일관적



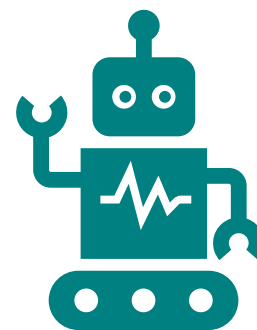
AI 에이전트와 잘 맞는 이유

- AI 에이전트의 작업 방식

- 코드 읽기 → 파일 수정 → 명령 실행 → 결과 확인
- 버튼 클릭 화면보다 명령어·텍스트 환경이 더 자연스러움

- TUI가 유리한 세 가지

- **자동화 용이** — 실행 과정이 텍스트로 남아 스크립트·파이프라인 연동이 용이
- **원격·클라우드 호환** — GUI 없이도 서버·컨테이너에서 그대로 동작
- **일관된 흐름** — 명령 → 실행 → 결과 → 수정 사이클이 AI 반복 작업과 정확히 일치
- → **Claude Code, Codex**가 명령줄을 적극 활용하는 이유



결론 — 실용적인 선택

- TUI의 부활은 복고가 아니다
 - 화려한 화면이 아니라, 사용자가 덜 생각하고 바로 일할 수 있는 일관성이 중요
- AI 시대의 인터페이스 기준
 - **단순성** — 사람과 AI 모두 혼란 없이 사용할 수 있는 구조
 - **자동화 가능성** — 스크립트·파이프라인으로 연결할 수 있는 열린 인터페이스
 - **일관성** — 앱이 달라도 동일한 방식으로 작동하는 예측 가능한 환경
- 사람과 AI가 함께 작업하기 쉬운 인터페이스가 AI 에이전트 시대의 더 큰 가치를 갖는다



가상환경Virtual Envs

- 가상환경virtualenvs
 - 파이썬에서 필요한 여러 라이브러리를 격리된 환경에 각각 여러 개 만들어서 사용하는 환경
 - 여러 라이브러리들이 서로 버전 의존성이 있으므로 각 용도에 맞게 환경을 여러 개 설정
 - nn : numpy + scipy + tensorflow + keras + pytorch
 - web : flask, Django



Conda 가상환경

- Conda cheat sheet
 - https://docs.conda.io/projects/conda/en/4.6.0/_downloads/52a95608c49671267e40c689e0bc00ca/conda-cheatsheet.pdf
- 가상환경 만들기: `conda create --name newenv python=3.10`
- 가상환경으로 들어가기: `conda activate newenv`
- 가상환경에서 나오기: `conda deactivate`
- 가상환경 리스트: `conda env list`
- 가상환경 지우기: `conda env remove --name newenv`
- 패키지 설치
 - `conda install jupyter`
 - `pip install jupyter`
- 패키지 삭제: `conda uninstall jupyter`
- 설치 패키지: `conda list`

가상환경 실습

- 적당한 이름으로 conda 가상 환경
 - Create
 - Activate, deactivate
 - Remove
 - Install numpy

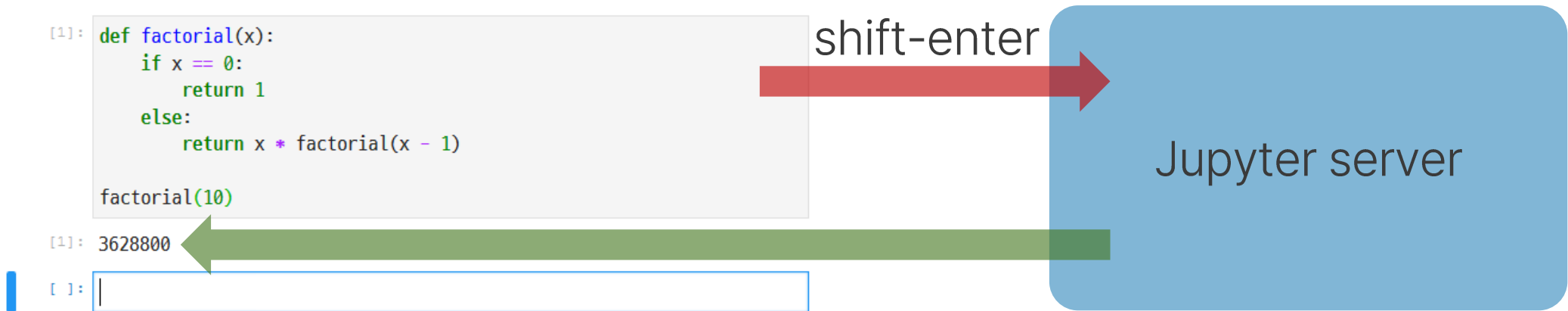
개발 환경: Jupyter

- Jupyter, Colab, etc

- 서버에 웹브라우저로 접속하여 코드를 입력하고 결과를 HTML로 돌려 받음
- 셀cell이라 부르는 코드 블록에 코드를 입력하고 shift-enter로 실행

- 설치: 가상환경이 켜진 프롬프트에서

- `pip install -r requirements.txt`



print()와 주석

- `print()`
 - 화면에 값을 출력
 - `print('hello python')`
- 주석
 - `#` 으로 시작하는 문장: 파이썬에서 해석되지 않음 무시
 - 소스 코드 설명
 - 코드 실행 방지
- 용도
 - 프로그램 분석, 실행 과정 추적
 - 버그 수정

변수와 자료형

- 변수 (Variable)
 - 값을 담는 그릇(이름). 언제든지 값을 바꿀 수 있음 (재할당)
 - 변수명 규칙: 영문, 숫자, _ 사용 (숫자로 시작 불가)
- 주요 자료형 (Data Types)
 - `int` 정수 (Integer) 예: `10`, `-5`, `0`
 - `float` 실수 (Floating-point) 예: `3.14`, `-0.01`
 - `str` 문자열 (String) 예: `"Hello"`, `'Python'`
 - `bool` 논리형 (Boolean) 예: `True`, `False`
- 타입 변환 (Casting)
 - `int()`, `float()`, `str()`, `bool()` 함수로 자료형 변환
 - `float(3) → 3.0` / `bool(0) → False` / `bool(1) → True`
 - `type()` 함수로 자료형 확인 예: `type(x) → <class 'int'>`

변수 선언 연습 Hands on Practice 01

- 상황
 - 생산 현장에서 관리하는 웨이퍼 정보(ID, 공정명, 수량, 불량 칩 수, 작업자명)를 Python 변수로 저장한다.
- 수강생 작성 내용
 - 웨이퍼 ID, 공정명, 웨이퍼 개수, 웨이퍼당 칩 개수, 불량 칩 개수, 작업자명을 각각 변수로 선언한다.
- 핵심 문법
 - 변수 선언 값을 담는 이름을 만들고 = 로 저장
 - 자료형 문자열(str), 정수(int), 실수(float)
 - 출력 print() 로 여러 변수를 한 번에 출력
- 주의
 - 저장되는 값은 실제 공정 데이터가 아닌 문법 학습용 예시값이다.

연산자

- 비교 연산자 (Comparison Operators)
 - 두 값을 비교해 **True** 또는 **False**를 반환한다. 조건문과 함께 사용한다.
 - **>** 왼쪽이 더 크면 **True**
 - **<** 왼쪽이 더 작으면 **True**
 - **>=** 왼쪽이 크거나 같으면 **True**
 - **<=** 왼쪽이 작거나 같으면 **True**
 - **==** 두 값이 같으면 **True**
 - **!=** 두 값이 다르면 **True**
- 논리 연산자 (Logical Operators)
 - 여러 조건을 조합할 때 사용한다.
 - **and** 두 조건이 모두 **True**일 때만 **True**
 - **or** 둘 중 하나라도 **True**이면 **True**
 - **not True** $\leftrightarrow$ **False** 반전
- 활용 예시
 - **score >= 60 and score < 100** $\rightarrow$ 60 이상 100 미만일 때 **True**
 - **age < 18 or age >= 65** $\rightarrow$ 미성년자 또는 고령자일 때 **True**

Walk-through

- 계산기
 - 두 수를 입력 받고 사칙 연산을 수행하는 프로그램

연산자 연습 Hands on Practice 02

- 상황
 - 웨이퍼 생산 상태를 판단하려면 전체 칩 수, 수율, 공정값 기준 만족 여부를 먼저 계산해야 한다.
- 수강생 작성 내용
 - 산술 전체 칩 수, 정상 칩 수, 불량률, 수율 계산
 - 비교 온도·압력이 기준 범위 안에 있는지 **True/False**로 확인
 - 논리 온도·압력·수율 조건을 **and** 로 묶어 최종 생산 가능 여부 판단
- 핵심 문법
 - 산술 연산자 **+** **-** ***** **/** **//** 비교 연산자 **>=** **<=** **/** 논리 연산자 **and**
- 주의
 - 이 단계에서는 **if** 문을 사용하지 않는다. 결과는 **True / False** 로 확인한다.
 - 기준값은 실제 공정 기준이 아닌 교육용 가상 기준이다.

리스트 (list)

- 핵심 개념

- 순서가 있는 자료형 — 인덱스(0부터)로 접근 가능
- 변경 가능 (Mutable) — 요소 추가, 수정, 삭제 자유
- 대괄호 [] 사용, 다양한 자료형 혼합 저장 가능

- 자주 쓰는 메서드

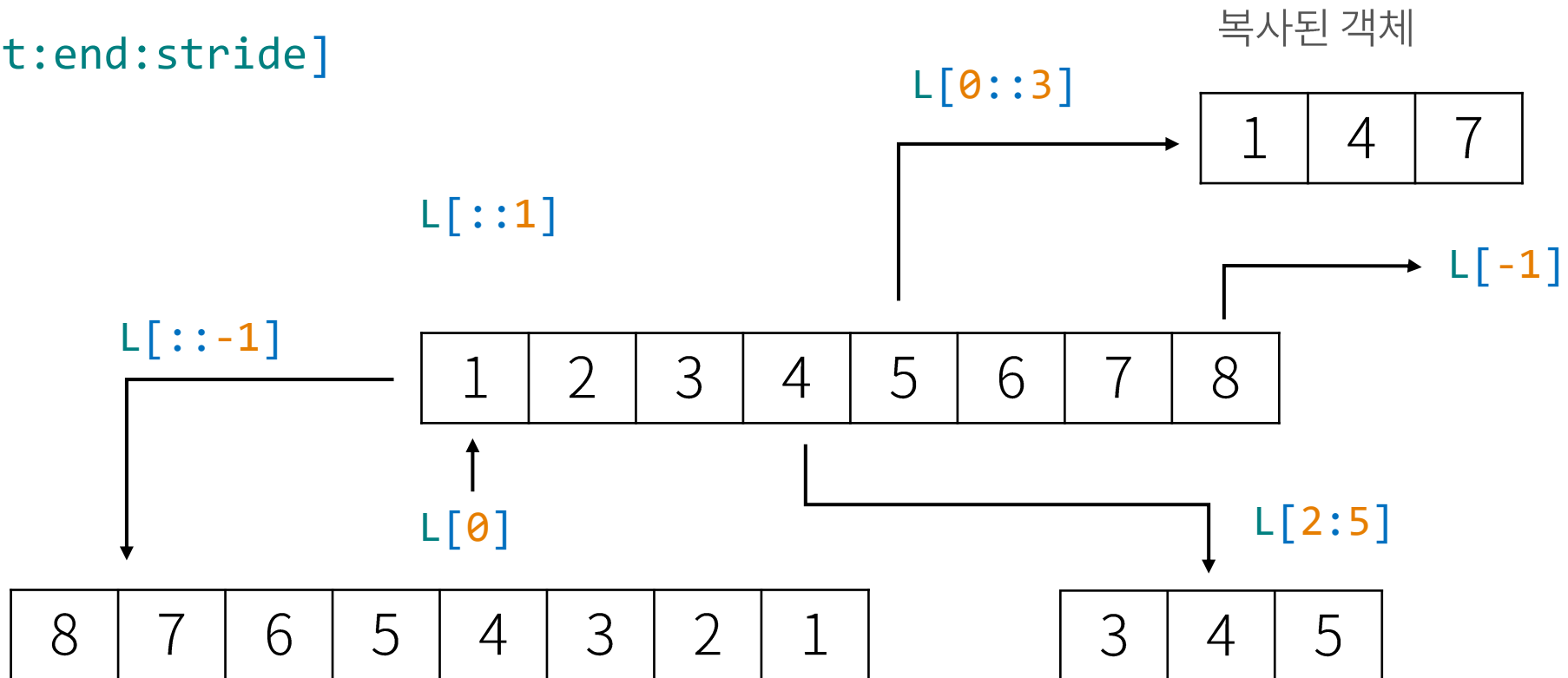
- `.append(x)` 리스트 끝에 x 추가
- `.sort()` 오름차순 정렬
- `.pop()` 마지막 요소 반환 및 삭제
- `len(x)` 리스트 길이(개수) 반환

- 인덱싱 & 슬라이싱

- `a[0]` → 첫 번째 요소 / `a[-1]` → 마지막 요소 / `a[1:3]` → 인덱스 1~2

슬라이싱Slicing

- `[i]` 형식으로 요소에 접근하는 방식의 확장
- 음수 index : `L[-1]`
- `[start:end:stride]`



Walk-through

- 다음 리스트에 `works` = ['회의록 정리', '보고서 작성']
 - 1. '출장' 추가
 - 2. '보고서 작성'을 '2팀 회의 참석'으로 변경

사전 (dict)

- **핵심 개념: Key-Value 쌍**
 - 데이터를 키(Key)와 값(Value) 쌍으로 저장하는 자료구조
 - 중괄호 {} 사용, 키를 통해 값을 빠르게 검색
- **기본 동작**
 - `d = {'k': 'v'}` 생성 후 `d['k']` 로 접근
 - `d['new_key'] = value` (없으면 추가, 있으면 수정)
 - `del d['k']` 또는 `d.pop('k')` 로 삭제
 - `d.get('k')` 키가 없어도 에러 없이 `None` 반환
- **유용한 메서드**
 - `keys()` 키 목록 `values()` 값 목록 `items()` (키, 값) 쌍

Walk-through

- 다음 사전에 `AI = {'ChatGPT': ['OpenAI', 5.5], 'Gemini': ['Google', 3.0], 'Claude': ['Anthropic', 4.7]}`
 - 1. `'Grok': ['xAI', 4.3]` 추가하기
 - 2. `'Claude'` 삭제 하기
 - 3. Gemini 버전을 3.1로 수정하기

자료구조 연습 Hands on Practice 03

- 상황
 - 웨이퍼 ID별로 재고 수량과 상태를 dict로 관리한다. 신규 등록, 조회, 수정, 삭제를 직접 구현한다.
- 수강생 작성 내용
 - Create 웨이퍼 ID를 key로 수량·상태 dict를 inventory에 추가
 - Read wafer_id가 inventory에 있으면 정보 출력
 - Update wafer_id가 있으면 수량과 상태를 입력값으로 수정
 - Delete wafer_id가 있으면 inventory에서 해당 항목 삭제
 - 각 단계에서 wafer_id 가 없는 경우는 추후 해결
- 핵심 문법
 - dict / key-value 구조 / 데이터 추가·수정·삭제
- 주의
 - 메뉴 처리와 반복 실행 구조는 제공 코드이며 수강생은 dict 조작 핵심 코드만 작성한다.

if 제어문

- 기본 구조

- `if` 조건: 실행문1 (조건 참)
- `elif` 조건2: 실행문2 (조건2 참)
- `else`: 실행문3 (모두 거짓)
- 주의: 콜론(:) 필수, 들여쓰기(`Tab` 또는 공백 4칸) 일치

- 비교 연산자

- `>` `<` `>=` `<=` `==` `!=`

- 논리 연산자

- `and` 두 조건이 모두 참
- `or` 두 조건 중 하나 참
- `not` 조건의 반대

- 예시

- `score = 85`
- `if score >= 60: print('합격') else: print('불합격')`

Walk-through

- BMI 계산기

- 키와 몸무게를 입력 받아 다음 공식으로 bmi를 계산하여 비만도를 등급으로 출력

$$\text{BMI} = \text{weight} / \text{height}^2$$



조건 활용 연습 Hands on Practice 04

- 상황
 - 연산자 실습에서 **True/False**로 계산했던 공정값 판단을, 이번에는 조건문으로 사람이 읽을 수 있는 상태 메시지로 바꾼다.
- 수강생 작성 내용
 - 온도 상태 기준 범위 안이면 '정상', 벗어나면 '기준 이탈'
 - 압력 상태 기준 범위 안이면 '정상', 벗어나면 '기준 이탈'
 - 수율 상태 기준 이상이면 '정상', 미달이면 '기준 미달'
 - 최종 판정 세 상태 모두 '정상'이면 '생산 계속 진행', 하나라도 아니면 '생산 중지 후 점검 필요'
- 핵심 문법
 - **if** / **elif** / **else** / 비교 연산자 / 논리 연산자 **and**
- 주의
 - 이 예제는 실제 장비 제어가 아닌 조건문 학습용 판정 프로그램이다.
 - 수율 계산 코드는 제공 코드이며, 수강생은 **if** 문만 작성한다.

for 반복문

- 기본 구조

- 순서가 있는 객체(리스트, 문자열 등)를 순회하며 코드 반복 실행
- `for` 변수 `in` 반복가능객체:
- 실행문

- `range()` 함수

- `range(n)` 0부터 `n-1`까지 (`n`번 반복)
- `range(a, b)` `a`부터 `b-1`까지
- `range(a, b, step)` `a`부터 `b-1`까지 `step` 간격으로

- 예시

- `for i in range(3): print(i)` → 출력: 0, 1, 2

자료구조: 리스트와 for

- 리스트는 **for**에 바로 사용 가능

```
for i in range(1, 6, 1):
```

```
for i in [1,2,3,4,5]:
```

```
items = ['a', 'b', 'c', ...]  
for item in items:  
    print(item)
```

Walk-through

- 성적처리

- 다음 두 리스트는 지수와 만수의 국어, 수학, 영어 점수를 저장하고 있다.
- 성적 처리를 위해 각 과목의 총점을 저장한 리스트를 만들어보자.

```
#      국어, 수학, 영어
jisoo = [90, 85, 93]
mansoo = [78, 92, 89]
```

Walk-through

- 행렬 덧셈

- 인덱싱을 이용해서 두 행렬의 덧셈을 수행하는 코드 만들기

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$C = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}$$

- 행렬 곱셈??

루프 활용 연습 Hands on Practice 05

- 상황
 - 1초마다 5건씩 들어오는 가상 공정 데이터를 배치 단위로 순회하며, 이상 의심 항목을 필터링해 별도 목록에 저장한다.
- 수강생 작성 내용
 - `batch_data`를 `for` 반복문으로 순회하며 아래 기준 중 하나라도 해당하면 `abnormal_data`에 추가
 - 불량 수 기준 `defect_count` 가 5 이상
 - 온도 기준 `temperature` 가 70 미만이거나 75 초과
- 핵심 문법
 - `list / dict / for / if / or / append()`
- 주의
 - 데이터 생성 함수와 출력 코드는 제공 코드이며 수강생은 필터링 루프만 작성한다.
 - 온도 범위와 불량 수 기준은 실제 품질 기준이 아닌 교육용 가상 기준이다.

객체와 점 연산자

- 객체 (Object)

- 값(데이터)과 그 값을 다루는 기능(메서드)이 함께 묶인 변수
- 문자열, 리스트, 사전 등은 모두 객체

- 점 연산자 (.)

- 객체에 있는 기능(메서드)을 불러내는 연산자
- 객체이름.메서드이름()

- 활용 예시

- "사과,바나나,오렌지".split(",") → ["사과", "바나나", "오렌지"]
- "::::".join(["쌀","베이컨","라면"]) → "쌀::::베이컨::::라면"
- [1,2,3].append(4) → [1, 2, 3, 4]

함수 정의와 사용법

- 함수 정의 (**def**)
 - **def** 키워드로 시작, 함수 이름과 매개변수 지정
 - **def** 함수명(매개변수): 실행문 **return** 반환값
- 매개변수 & 반환값
 - 매개변수: 함수 내부로 전달되는 값
 - **return** 으로 결과를 외부로 내보냄
- 함수 호출 (**Call**)
 - 정의된 함수는 함수명() 형태로 호출해야 실행됨
- 예시
 - **def** add_numbers(a, b): **return** a + b
 - **total** = add_numbers(3, 5) → **print**(total) 출력: 8

Walk-through

- 평균을 구하는 함수 1
 - 숫자로 채워진 리스트를 입력 받아 평균을 구하는 함수

함수 활용 연습 Hands on Practice 06

- 상황
 - 1초마다 무작위 생산 데이터가 10회 들어온다고 가정하고, 매번 같은 수율 계산 함수를 재사용해 결과를 출력한다.
- 수강생 작성 내용
 - `calculate_yield(wafer_count, chips_per_wafer, defective_chips)` 함수 내부를 완성한다.
 - 1단계 전체 칩 수 = 웨이퍼 개수 × 웨이퍼당 칩 개수
 - 2단계 정상 칩 수 = 전체 칩 수 - 불량 칩 수
 - 3단계 수율 = 정상 칩 수 ÷ 전체 칩 수 × 100
 - 4단계 계산된 수율을 `return` 으로 반환
- 핵심 문법
 - `def` / 매개변수 / `return` / 산술 연산자 / 함수 호출
- 주의
 - 수율 계산식은 학습용 단순화 공식이며 실제 현업의 수율 정의와 다를 수 있다.
 - 데이터 생성 및 반복 호출 코드는 제공 코드이며 수강생은 함수 내부만 작성한다.

모듈 사용법

- 모듈 가져오기 (**Import**) 방식
 - **import math** 모듈 전체 가져오기
 - **import pandas as pd** 별칭(Alias) 사용 — 긴 이름 줄이기
 - **from random import randint** 특정 기능만 가져오기
- 주요 모듈
 - pandas 데이터 분석, 처리 (**import pandas as pd**)
 - numpy 수치 연산, 배열 (**import numpy as np**)
 - matplotlib 데이터 시각화 (**import matplotlib.pyplot as plt**)
 - scikit-learn 머신러닝 모델링 (**import sklearn**)

Walk-through

- 로또 번호 생성기
 - 호출 하면 로또 번호 6개를 반환하는 함수 작성
 - 사용자로 부터 게임 수를 입력받고 입력 받은 수만큼 함수를 반복 호출하여 여러 개의 번호를 생성

Walk-through

- 표준편차를 구하는 함수

- 숫자로 채워진 리스트를 입력 받아 평균과 표준편차를 구하는 함수
- math 모듈의 sqrt() 활용

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=k}^N (x_k - m)^2}$$

에러 대응의 정석

- **에러 메시지를 꼼꼼히 살펴보는 것이 최선입니다.**
- **에러 메시지 읽는 법**
 - 가장 마지막 줄부터 읽기 — 에러 종류와 원인이 거기 있음
 - 에러 타입 확인 — `NameError`, `SyntaxError` 등 '무슨 문제인지' 파악
 - 발생 위치 확인 — "line 3" 처럼 몇 번째 줄에서 생겼는지 확인
 - 위아래 문맥 확인 — 오타 없으면 바로 윗줄의 괄호/따옴표 미종료 의심
- **해결이 안 될 때**
 - 에러 메시지 마지막 줄을 그대로 복사해서 검색하거나 AI에게 질문
- **에러 메시지 구조 예시**
 - `Traceback (most recent call last): File "ex.py", line 3`
 - `NameError: name 'result' is not defined`

ModuleNotFoundError

- 원인
 - `import` 하려는 모듈을 파이썬이 찾을 수 없을 때 발생
 - 모듈 미설치 또는 모듈 이름 오타 (대소문자 포함)
- 발생 예시
 - `import pandas` → `ModuleNotFoundError: No module named 'pandas'`
- 해결 방법
 - 1. 모듈 설치: `pip install pandas` (터미널) 또는 `!pip install pandas` (주피터)
 - 2. 철자 확인: `import Pendes(X)` → `import pandas(0)`
 - 설치 목록 확인: `!pip list`

SyntaxError

- 원인
 - 파이썬이 이해할 수 없는 문법 — 실행 전 파싱 단계에서 발생
 - 오타, 기호 누락, 잘못된 구조가 주요 원인
- 자주 발생하는 상황
 - `if x > 0 print(x)` 콜론(:) 누락
 - `print('Hello'` 닫는 괄호 누락
 - `msg = "Python` 닫는 따옴표 누락
- 해결 방법
 - 에러 메시지의 ^ 기호가 가리키는 지점 또는 바로 윗줄 확인
 - 괄호/따옴표 짝 확인 — 열면 반드시 닫기
 - `if, for, while, def, class` 뒤에는 반드시 콜론(:)

IndentationError

- 원인
 - 들여쓰기가 일치하지 않을 때 발생
 - 파이썬은 들여쓰기로 코드 블록 구분 — 탭과 스페이스 혼용 금지
- 자주 발생하는 상황
 - 같은 블록인데 들여쓰기 깊이가 다름 (2칸 vs 4칸)
 - 탭과 스페이스를 혼용
 - 함수/조건문 블록 내부에 들여쓰기 누락
- 해결 방법
 - 스페이스 4칸을 일관되게 사용 (PEP 8 권장)
 - 에디터 설정에서 "탭을 스페이스로 변환" 옵션 활성화
 - 같은 코드 블록 안의 모든 줄은 동일한 들여쓰기 깊이 유지

NameError

- 원인
 - 파이썬이 코드 실행 중 모르는 이름(변수, 함수)을 발견했을 때
- 주요 원인
 - 변수나 함수를 정의하기 전에 사용
 - 변수명/함수명 철자(Typo) 오류
 - 대소문자 혼동 (a와 A는 다른 변수)
- 발생 예시
 - `print(score)` `score` 변수를 만든 적 없음 → `NameError`
 - `prnit(data)` `print` 오타 → `NameError`
- 해결 방법
 - 사용 전 변수에 값 할당 확인, 함수명/변수명 철자 재확인

TypeError

- 원인
 - 연산이나 함수가 부적절한 데이터 타입에 적용될 때
 - 서로 다른 타입끼리 연산, 잘못된 타입의 인자 전달
- 발생 예시
 - `age = 25; print("나이: " + age)` → `TypeError: str + int` 불가
- 해결 방법
 - 1. 타입 변환: `print("나이: " + str(age))`
 - 2. f-string 사용 (권장): `print(f"나이: {age}")`
- 타입 확인 방법
 - `type`(변수명) 함수로 현재 데이터 타입 확인